UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 07 • **Ejercicio 06** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 6: Videojuego / Simulador de Combate Táctico

Contexto

En este ejercicio aprenderemos a diseñar un sistema combinando **Clases Abstractas** para modelar la identidad y el estado vital compartido de una entidad central ("es-Un"), con **Interfaces** para incorporar capacidades y modificadores transversales mediante el principio **"Favor Composition over Inheritance"** (Composición sobre Herencia).

Asimismo, profundizaremos en el rol del **Tipo Aparente** frente al **Tipo Real** y el aprovechamiento del **despacho dinámico (polimorfismo puro)** para orquestar la arena de combate sin depender de comprobaciones manuales de tipo (`instanceof`) ni conversiones forzadas (*downcasting*).

Enunciado

Se solicita diseñar el núcleo lógico de un simulador de combate táctico por turnos en Java, prescindiendo de complejidades espaciales o coordenadas cartesianas para enfocarse exclusivamente en el modelado orientado a objetos, el polimorfismo y la composición limpia.

1. Jerarquía de Combatientes (`Personaje`)

Todos los combatientes del simulador comparten una estructura vital común modelada en la clase abstracta **`Personaje`**. Encapsula el nombre y los puntos de vida del combatiente, proveyendo métodos concretos para:

- `recibirDanio(int cantidad)`: Reduce los puntos de vida, garantizando que nunca queden por debajo de 0.
- `curar(int cantidad)`: Restaura vitalidad incrementando los puntos de vida.
- `estaVivo(): boolean`: Indica si el personaje continúa en pie (`puntosVida > 0`).
- Getters para `nombre` y `puntosVida`.

Asimismo, define el contrato abstracto ofensivo que cada arquetipo debe especializar:

```
public abstract void atacar(Personaje objetivo);
```

Se deben diseñar las siguientes especializaciones concretas:

- **`Guerrero`**: Combatiente cuerpo a cuerpo con `fuerzaFisica: int`. Al atacar, inflige daño directo equivalente a dicha fuerza sobre el objetivo.
- **`Mago`**: Combatiente arcano con `mana: int`, `costoHechizo: int` (ej. 15) y `danioMagico: int` (ej. 20). Al atacar, comprueba si dispone de maná suficiente: de ser así, consume el costo del hechizo e inflige el daño mágico; en caso contrario, el ataque falla (sin consumir maná ni dañar al adversario).
- **`Arquero`**: Combatiente ágil con `danioBase: int` y `destreza: int`. Al atacar, inflige un daño calculado como la suma directa de su daño base más su destreza.

2. Desafío de Diseño: Modificadores Dinámicos (Composición sobre Herencia)

El juego requiere que los personajes puedan adoptar efectos secundarios en sus ataques (como envenenar) o equiparse con ítems protectores (como amuletos).

> **Alerta de Diseño:** Si utilizáramos herencia para cada combinación, sufriríamos una explosión de clases inmanejable (`GuerreroVenenoso` , `ArqueroConAmuleto` , `MagoVenenosoConAmuleto`). Aplique el principio **"Favor Composition over Inheritance"** mediante contratos de interfaz:

- **Modificador de Ataque (`EfectoAtaque`):**
  - Defina la interfaz `EfectoAtaque` con el método `void aplicar(Personaje objetivo)` .
  - Implemente la clase `Veneno` (`danioVeneno: int`), que inflige daño residual sobre el adversario al aplicarse.
  - Permita que la clase abstracta `Personaje` componga opcionalmente una referencia a `EfectoAtaque` (inyectable mediante `setEfectoAtaque` y consultable con `getEfectoAtaque`). Al atacar, luego de aplicar el daño propio del arquetipo, si el personaje posee un efecto configurado y el objetivo sigue vivo, debe dispararse el efecto sobre él.
- **Ítems Transversales (`Equipable`):**
  - Defina la interfaz `Equipable` con el método `void equipar(Personaje p)` .
  - Implemente la clase `ItemEquipamiento` (por ejemplo, con un atributo `bonoVida: int`), que al equiparse sobre un personaje cure inmediatamente dicha cantidad de vida.

3. Orquestador de Batalla (`MotorCombate`)

La clase `MotorCombate` administra una colección de combatientes (`List<Personaje> arena`). Debe interactuar puramente contra la abstracción `Personaje` y proveer:

- `agregarCombatiente(Personaje p)` : Registra un nuevo combatiente en la arena.
- `ejecutarRondaCombate()` : Recorre a los combatientes registrados. Cada combatiente vivo busca a un adversario vivo distinto de sí mismo y ejecuta polimórficamente su ataque.
- `obtenerSobrevivientes(): List<Personaje>` : Retorna una lista filtrada conteniendo exclusivamente a los combatientes que aún permanecen con vida.

Tareas:

1. Diseñe las interfaces `EfectoAtaque` y `Equipable`.
2. Implemente las clases `Veneno` (que cumple `EfectoAtaque`) e `ItemEquipamiento` (que cumple `Equipable`).
3. Diseñe la clase abstracta `Personaje` con el estado protegido común, métodos de vida, composición con `EfectoAtaque` y el método abstracto `atacar(Personaje objetivo)`.
4. Implemente las subclases `Guerrero`, `Mago` y `Arquero`, respetando sus mecánicas específicas y disparando el efecto secundario si corresponde.
5. Diseñe la clase `MotorCombate` interactuando exclusivamente con el tipo aparente `Personaje`.

Restricciones y Tips

- **Composición sobre Herencia:** No cree clases como `GuerreroVenenoso` o `ArqueroConAmuleto`. Los modificadores y efectos deben asociarse dinámicamente mediante referencias a las interfaces (`setEfectoAtaque(...)`, `equipar(...)`).
- **Polimorfismo en la Arena:** El `MotorCombate` no debe utilizar sentencias condicionales (`if (p instanceof Guerrero)`) ni conversiones forzadas (*downcasting*) para ejecutar el combate. La llamada `atacante.atacar(objetivo)` debe resolverse puramente mediante despacho dinámico en tiempo de ejecución.
- **Invariante de Puntos de Vida:** Los puntos de vida de un personaje jamás deben quedar con un valor negativo; si el daño supera la vida restante, debe establecerse en 0.
- **Fallo del Mago:** Cuando el mago no posee maná suficiente para costear el hechizo, el ataque no debe consumir maná ni alterar la vida del adversario ni aplicar el efecto secundario.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué `Personaje` debe modelarse como una clase abstracta en lugar de una interfaz o una clase concreta instanciable?
2. Si quisiéramos que un personaje pueda ser envenenado, congelado o quemado en simultáneo o alternadamente, ¿qué ocurriría si hubiéramos utilizado herencia en lugar de componer con la interfaz `EfectoAtaque`? ¿Qué principio de diseño se aplica aquí?
3. ¿De qué manera el diseño de `MotorCombate` respeta el Principio Abierto/Cerrado (OCP) al momento de incorporar una nueva clase de combatiente (como un `Nigromante`)?